

# 4

## 과제 #4

### Q1.



#### Task 1. PE 파일 패치

Assignment 03과 함께 제공된 abex\_crackme1.exe를 크랙하는 방법은 다양하다. 이번 과제에서는 401026h 메모리 번지에 탑재된 JE 40103D(74h 15h) 명령을 JNE 40103D (75h 15h)명령으로 어셈블하여 프로그램을 공략하기로 하자.

디버거를 사용해 이미지(메모리에 적재된 PE 파일) 내 CODE 섹션에 수정을 가하더라도, 디버깅을 재시작하거나 프로그램을 더블클릭으로 단독 실행하면 상기 opcode JNE가 JE로 되돌아간다.

프로그램을 영구적으로 크래킹하기 위해서는 HxD와 같은 hex editor로 abex\_crackme1.exe 파일을 열고 파일에 직접 수정을 가해야 한다. 이 때, PE 파일에 저장된 JE 40103D 명령(74h 15h)을 찾기 위해서는 RVA to RAW 연산을 수행해야 한다.

아래 사항을 간단히 정리하시오.

- 변경하려는 데이터가 메모리 상 VA=401026h (RVA=1026h)에 있다면, RAW 주소는?
- RAW 주소를 구하기 위한 계산 과정
- 프로그램 패치 과정 (HxD 등 hex 에디터 화면을 수정 전후로 캡처)
- 패치한 프로그램의 실행 결과



어셈블한다 = 명령어를 변경한다.

### 1. 변경하려는 데이터가 메모리 상 VA=401026h (RVA=1026h)에 있다면, RAW 주소와 계산 과정

| Name                     | Offset | Size | RAW      | RVA      | Value    |
|--------------------------|--------|------|----------|----------|----------|
| Name                     | 00     | 08   | 000001F8 |          |          |
| PhysicalAddressOrVirtual | 00     | 04   | 00000200 |          | 00001000 |
| VirtualAddress           | 0C     | 04   | 00000204 | 00001000 |          |
| PointerToRawData         | 14     | 04   | 0000020C | 00000E00 |          |
| PointerToRelocations     | 18     | 04   | 00000210 | 00000E00 |          |
| PointerToLinenumbers     | 1C     | 04   | 00000214 | 00000000 |          |
| NumberOfRelocations      | 20     | 02   | 00000218 | 0000     |          |
| NumberOfLinenumbers      | 22     | 02   | 0000021A | 0000     |          |
| Characteristics          | 24     | 04   | 0000021C | 60000020 |          |

$$\text{RAW} = (\text{RVA} - \text{VirtualAddress}) + \text{PointerToRawData}$$

$$= (0x1026 - 0x1000) + 0x0600$$

$$= 0x0026 + 0x0600$$

$$= 0x0626$$

## 2. 프로그램 패치 과정과 결과

### 기존 프로그램

|          |       |     |      |    |                             |    |     |    |    |
|----------|-------|-----|------|----|-----------------------------|----|-----|----|----|
| 메모리 맵    | 호출 스택 | SEH | 스크립트 | 기호 | 소스                          | 참조 | 스레드 | 핸들 | 추적 |
| 00401026 | 74 15 |     |      |    | je abex_crackme1.exe.40103D |    |     |    |    |
| 00401028 | 6A 00 |     |      |    | nush 0                      |    |     |    |    |

#### 2-1. RAW = 0x0626으로 이동

이동

오프셋(O):

626

☒ 16진수

☐ 10진수

☐ 8진수

오프셋 연결 대상

☒ 처음

☐ 현재 오프셋

☐ 마지막(뒤로)

수락

취소

|          |                                                 |                  |
|----------|-------------------------------------------------|------------------|
| 000005C0 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | .....            |
| 000005D0 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | .....            |
| 000005E0 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | .....            |
| 000005F0 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | .....            |
| 00000600 | 6A 00 68 00 20 40 00 68 12 20 40 00 6A 00 E8 4E | j.h. @.h. @.j.èN |
| 00000610 | 00 00 00 68 94 20 40 00 E8 38 00 00 00 33 F6 46 | ...h" @.è8...3øF |
| 00000620 | 46 46 46 48 3B C6 74 15 6A 00 68 35 20 40 00 68 | FFFH;Eu.j.h5 @.h |

오프셋(h): 626

뒤어쓰기

- 맨 밑에 74 15 → JE 명령어(74 15)

## 2-2. 74 (JE 명령어) → 75 (JNE 명령어) 로 변경

|          |                                                 |                  |
|----------|-------------------------------------------------|------------------|
| 00000600 | 6A 00 68 00 20 40 00 68 12 20 40 00 6A 00 E8 4E | j.h. @.h. @.j.èN |
| 00000610 | 00 00 00 68 94 20 40 00 E8 38 00 00 00 33 F6 46 | ...h" @.è8...3øF |
| 00000620 | 46 46 46 48 3B C6 75 15 6A 00 68 35 20 40 00 68 | FFFH;Eu.j.h5 @.h |

오프셋(h): 627

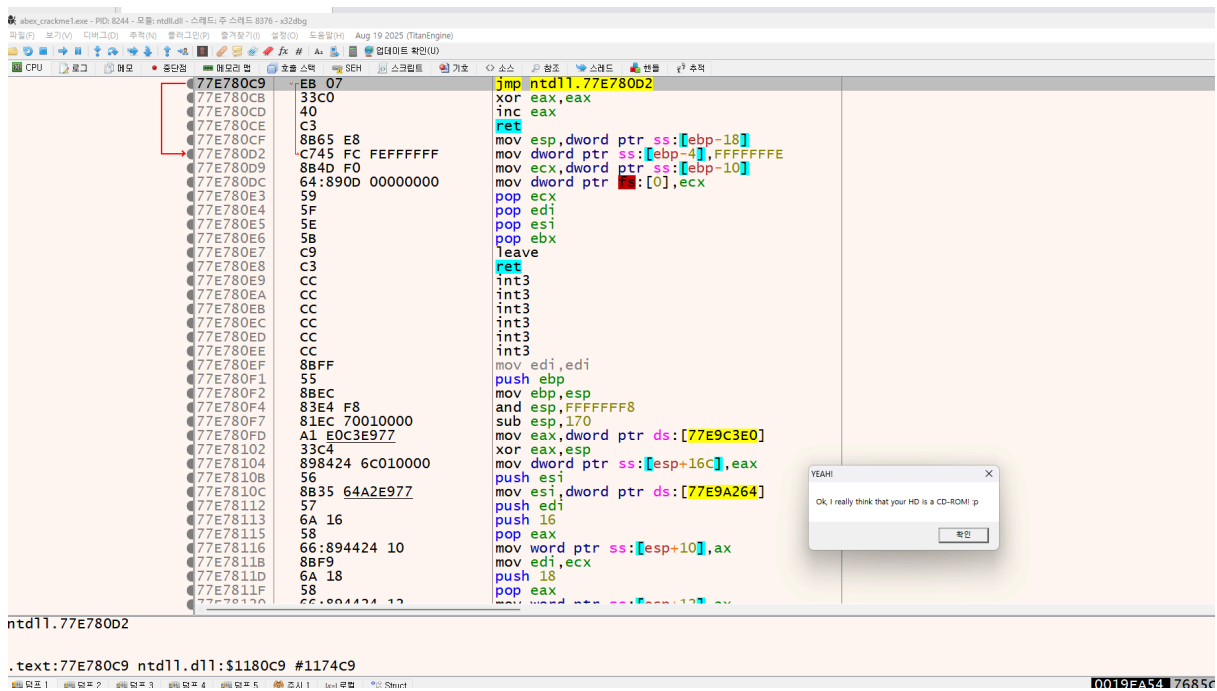
\* 변경됨 \*

뒤어

## 2-3. 수정 결과

|          |       |                          |
|----------|-------|--------------------------|
| 00401024 | 3BC6  | cmp eax,esi              |
| 00401026 | 75 15 | jne abex_crackme1.40103D |
| 00401028 | 6A 00 | push 0                   |

## 2-4. 실행 결과



## Q2.



### Task 2. PE 파서 제작

첨부된 소스코드—PE\_parser.cpp—를 기반으로, 콘솔 기반의 x86 PE 파서를 제작하고 실행 결과와 소스코드를 제출하라. 제작한 프로그램을 검증하면서 최소 2개의 x86 PE 파일을 파싱해 보라. 그 중 하나의 PE 파일을 abex\_crackme1.exe로 하고 그 결과가 아래와 같은지 검증하라. 섹션 정보, Import Address Table (IAT), IAT에 내재된 함수 이름을 파싱할 수 있어야 한다.

#### • 실행예시

File size=8192 bytes

File signature=MZ

Offset to the NT header=0x100

OptionalHeader.BaseOfCode=0x1000

OptionalHeader.SizeOfCode=0x200

OptionalHeader.AddressOfEntryPoint=0x1000

OptionalHeader.BaseOfData=0x2000  
OptionalHeader.ImageBase=0x400000

### SECTION INFORMATION ###

1번째 section: CODE

PointerToRawData: 0x600

SizeOfRawData: 0x200

VirtualAddress: 0x1000

VirtualSize: 0x1000

2번째 section: DATA

PointerToRawData: 0x800

SizeOfRawData: 0x200

VirtualAddress: 0x2000

VirtualSize: 0x1000

3번째 section: .idata

PointerToRawData: 0xa00

SizeOfRawData: 0x200

VirtualAddress: 0x3000

VirtualSize: 0x1000

4번째 section: .reloc

PointerToRawData: 0xc00

SizeOfRawData: 0x200

VirtualAddress: 0x4000

VirtualSize: 0x1000

5번째 section: .rsrc

PointerToRawData: 0xe00

SizeOfRawData: 0x400

VirtualAddress: 0x5000

VirtualSize: 0x1000

### IAT ###

IAT가 저장된 섹션: .idata

RVA to RAW: 0x3000→0xa00

ImportDescriptor[0].Name=KERNEL32.dll

- function name (RVA=0x307c), GetDriveTypeA
- function name (RVA=0x308c), ExitProcess
- ImportDescriptor[1].Name=USER32.dll
- function name (RVA=0x309a), MessageBoxA



파싱 = 파일 안의 데이터를 규칙에 맞게 나누어 읽어오는 과정

## 1. 소스코드 #1 ( **PEParser\_1.cpp** )

```
#include <Windows.h>
#include <stdio.h>

int main(void)
{
    wchar_t path_pefile[] = L"C:\\Users\\백다현\\Desktop\\abex_crackme1.exe
\\abex_crackme1.exe";

    HANDLE hFile = NULL, hFileMap = NULL;
    LPBYTE lpFileBase = NULL;
    DWORD dwSize = 0;

    PIMAGE_DOS_HEADER pDosHeader = NULL;
    PIMAGE_NT_HEADERS pNtHeader = NULL;

    hFile = CreateFileW(path_pefile, GENERIC_READ, FILE_SHARE_READ, NU
LL, OPEN_EXISTING, FILE_ATTRIBUTE_NORMAL, NULL);
    if (hFile == INVALID_HANDLE_VALUE)
    {
        printf("CreateFileW() failed. Error code=%lu\n", GetLastError());
        return GetLastError();
    }
    dwSize = GetFileSize(hFile, 0);
    printf("File size=%lu bytes\n\n", dwSize);

    hFileMap = CreateFileMapping(hFile, NULL, PAGE_READONLY, 0, 0, NUL
```

```

L);
// lpFileBase 포인터는 OS에 의해 메모리에 로드된 PE 파일의 가장 첫 바이트를
가리킴.
//lpFileBase가 NULL이라면 MapViewOfFile() 함수가 실패했다는 의미이다. 여
기서는 예외 처리를 생략한다.
lpFileBase = (LPBYTE)MapViewOfFile(hFileMap, FILE_MAP_READ, 0, 0, d
wSize);

printf("File signature=%c%c\n", lpFileBase[0], lpFileBase[1]);

pDosHeader = (PIMAGE_DOS_HEADER)lpFileBase;
printf("Offset to the NT header=%#x\n\n", pDosHeader->e_lfanew);

pNtHeader = (PIMAGE_NT_HEADERS)(lpFileBase + pDosHeader->e_lfan
ew);
printf("OptionalHeader.BaseOfCode=%#x\n", pNtHeader->OptionalHead
er.BaseOfCode);
printf("OptionalHeader.SizeOfCode=%#x\n", pNtHeader->OptionalHeade
r.SizeOfCode);
printf("OptionalHeader.AddressOfEntryPoint=%#x\n", pNtHeader->Optio
nalHeader.AddressOfEntryPoint);
printf("OptionalHeader.BaseOfData=%#x\n", pNtHeader->OptionalHeade
r.BaseOfData);
printf("OptionalHeader.ImageBase=%#x\n\n", pNtHeader->OptionalHead
er.ImageBase);

printf("### SECTION INFORMATION ###\n");
/*TODO: 여기서부터 코딩 시작*/
// -----
// [1] 섹션 테이블 출력
// -----
PIMAGE_FILE_HEADER pFileHeader = &pNtHeader->FileHeader;
WORD numberOfSections = pFileHeader->NumberOfSections;

// IMAGE_FIRST_SECTION(pNtHeader) 매크로 대신 수동 계산
PIMAGE_SECTION_HEADER pSection = (PIMAGE_SECTION_HEADER)(
(LPBYTE)&pNtHeader->OptionalHeader + pNtHeader->FileHeader.Size
OfOptionalHeader

```

```

);

for (WORD i = 0; i < numberOfSections; ++i) {
    char name[9] = { 0 };
    memcpy(name, pSection[i].Name, 8);

    printf("%d번째 section: %s\n", i + 1, name);
    printf("PointerToRawData: %#x\n", pSection[i].PointerToRawData);
    printf("SizeOfRawData: %#x\n", pSection[i].SizeOfRawData);
    printf("VirtualAddress: %#x\n", pSection[i].VirtualAddress);
    printf("VirtualSize: %#x\n", pSection[i].Misc.VirtualSize);
}

// -----
// [2] IAT(Import Directory) 파싱
// -----
printf("### IAT ###\n");

IMAGE_DATA_DIRECTORY importDir =
    pNtHeader->OptionalHeader.DataDirectory[IMAGE_DIRECTORY_ENTRY_IMPORT]; // index 1

if (importDir.VirtualAddress == 0 || importDir.Size == 0) {
    printf("No Import Directory found.\n");
}
else {
    // (a) IAT가 저장된 섹션 찾기 + RVA→RAW 계산 함수 없이 인라인 처리
    // importDir.VirtualAddress 가 속하는 섹션을 찾아 RAW 오프셋 계산
    int importRaw = -1;
    char containingSection[9] = "Unknown";
    for (WORD i = 0; i < numberOfSections; ++i) {
        DWORD va = pSection[i].VirtualAddress;
        DWORD vsize = pSection[i].Misc.VirtualSize ? pSection[i].Misc.VirtualSize : pSection[i].SizeOfRawData;

        if (importDir.VirtualAddress >= va && importDir.VirtualAddress < va + vsize) {
            importRaw = (int)(pSection[i].PointerToRawData + (importDir.VirtualAddress - va));
        }
    }
}

```

```

alAddress - va));
    memset(containingSection, 0, sizeof(containingSection));
    memcpy(containingSection, pSection[i].Name, 8);
    break;
}
}

// 헤더 영역일 가능성: RVA < SizeOfHeaders 이면 RAW==RVA 로 취급
if (importRaw < 0 && importDir.VirtualAddress < pNtHeader->Optional
Header.SizeOfHeaders) {
    importRaw = (int)importDir.VirtualAddress;
    strcpy_s(containingSection, sizeof(containingSection), "Headers");
}

printf("IAT가 저장된 섹션: %s\n", containingSection);
if (importRaw >= 0)
    printf("RVA to RAW: %#x→0x%x\n", importDir.VirtualAddress, impo
rtRaw);
else
    printf("RVA to RAW 변환 실패 (IAT 위치 계산 불가)\n");

if (importRaw >= 0 && (DWORD)importRaw < dwSize) {
    PIMAGE_IMPORT_DESCRIPTOR pDesc = (PIMAGE_IMPORT_DESCRI
PTOR)(lpFileBase + importRaw);

    // (b) ImportDescriptor 배열 순회 (모두 0이면 종료)
    for (int idx = 0; ; ++idx, ++pDesc) {
        // 안전 범위 체크
        SIZE_T curOff = (SIZE_T)((LPBYTE)pDesc - lpFileBase);
        if (curOff + sizeof(IMAGE_IMPORT_DESCRIPTOR) > dwSize) brea
k;

        if (pDesc->OriginalFirstThunk == 0 &&
            pDesc->FirstThunk == 0 &&
            pDesc->Name == 0 &&
            pDesc->TimeDateStamp == 0 &&
            pDesc->ForwarderChain == 0) {
            break; // 끝

```

```

    }

    // DLL 이름: pDesc→Name (RVA) → RAW 변환 (인라인)
    int nameRaw = -1;
    if (pDesc→Name < pNtHeader→OptionalHeader.SizeOfHeaders) {
        nameRaw = (int)pDesc→Name;
    }
    else {
        for (WORD i = 0; i < numberOfSections; ++i) {
            DWORD va = pSection[i].VirtualAddress;
            DWORD vsize = pSection[i].Misc.VirtualSize ? pSection[i].Misc.VirtualSize : pSection[i].SizeOfRawData;
            if (pDesc→Name >= va && pDesc→Name < va + vsize) {
                nameRaw = (int)(pSection[i].PointerToRawData + (pDesc→Name - va));
                break;
            }
        }
    }

    char dllName[512] = { 0 };
    if (nameRaw >= 0 && (DWORD)nameRaw < dwSize) {
        // 안전 문자열 복사 (최대 길이는 맵 끝까지)
        strncpy_s(dllName, sizeof(dllName), (const char*)(lpFileBase + nameRaw), _TRUNCATE);
    }
    else {
        strcpy_s(dllName, sizeof(dllName), "<invalid>");
    }
    printf("ImportDescriptor[%d].Name=%s\n", idx, dllName);

    // (c) Thunk 테이블: OriginalFirstThunk(있으면) 우선, 없으면 FirstThunk 사용
    DWORD thunkRVA = pDesc→OriginalFirstThunk ? pDesc→OriginalFirstThunk : pDesc→FirstThunk;

    // thunkRVA → RAW 변환 (인라인)
    int thunkRaw = -1;

```

```

    if (thunkRVA < pNtHeader->OptionalHeader.SizeOfHeaders) {
        thunkRaw = (int)thunkRVA;
    }
    else {
        for (WORD i = 0; i < numberOfSections; ++i) {
            DWORD va = pSection[i].VirtualAddress;
            DWORD vsize = pSection[i].Misc.VirtualSize ? pSection[i].Misc.VirtualSize : pSection[i].SizeOfRawData;
            if (thunkRVA >= va && thunkRVA < va + vsize) {
                thunkRaw = (int)(pSection[i].PointerToRawData + (thunkRVA - va));
                break;
            }
        }
    }

    if (thunkRaw < 0 || (DWORD)thunkRaw >= dwSize) {
        printf(" Thunk RVA → RAW 변환 실패\n");
        continue;
    }

    // (d) 개별 thunk 순회 (32비트: 4바이트씩, 0이면 종료)
    SIZE_T off = (SIZE_T)thunkRaw;
    while (off + 4 <= dwSize) {
        DWORD entry = 0;
        memcpy(&entry, lpFileBase + off, 4);
        if (entry == 0) break;

        if (entry & 0x80000000u) {
            // ordinal import
            WORD ord = (WORD)(entry & 0xFFFF);
            printf("- ordinal: %u\n", ord);
        }
        else {
            // name import: entry는 IMAGE_IMPORT_BY_NAME 구조체의 RVA
            int ibnRaw = -1;
            if (entry < pNtHeader->OptionalHeader.SizeOfHeaders) {

```

```

        ibnRaw = (int)entry;
    }
    else {
        for (WORD i = 0; i < numberOfSections; ++i) {
            DWORD va = pSection[i].VirtualAddress;
            DWORD vsize = pSection[i].Misc.VirtualSize ? pSection
[i].Misc.VirtualSize : pSection[i].SizeOfRawData;
            if (entry >= va && entry < va + vsize) {
                ibnRaw = (int)(pSection[i].PointerToRawData + (entry
- va));

                break;
            }
        }
    }

    if (ibnRaw >= 0 && (DWORD)(ibnRaw + 2) < dwSize) {
        // [2바이트 hint] + 함수명(ASCII, null-terminated)
        const char* fname = (const char*)(lpFileBase + ibnRaw +
2);

        printf("- function name (RVA=%#x), %s\n", entry, fname);
    }
    else {
        printf("- function name (RVA=%#x), <cannot read>\n", ent
ry);
    }
}
off += 4;
}
}
}
}

```

```
/*TODO: 여기까지 코딩*/
```

```
/*Windows로부터 할당받은 리소스를 역순으로 반환*/
UnmapViewOfFile(lpFileBase);
```

```

    CloseHandle(hFileMap);
    CloseHandle(hFile);
    /*main() 함수가 끝까지 실행되었음을 알리기 위해 0을 반환*/
    return 0;
}

```

The screenshot shows the 'File size=8192 bytes' and 'File signature=MZ' information. It also displays the 'OptionalHeader' fields, including 'BaseOfCode=0x1000', 'SizeOfCode=0x200', 'AddressOfEntryPoint=0x1000', 'BaseOfData=0x2000', and 'ImageBase=0x400000'. The 'SECTION INFORMATION' section lists five sections: CODE, DATA, .idata, .reloc, and .rsrc, each with its respective pointer, size, and virtual address. The 'IAT' section shows the 'ImportDescriptor' table, which includes the 'Name' of the DLL and the 'Name' of the function being imported. The IAT entries are: 'KERNEL32.dll' with functions 'GetDriveTypeA' and 'ExitProcess', and 'USER32.dll' with the function 'MessageBoxA'.

```

File size=8192 bytes
File signature=MZ
Offset to the NT header=0x100
OptionalHeader.BaseOfCode=0x1000
OptionalHeader.SizeOfCode=0x200
OptionalHeader.AddressOfEntryPoint=0x1000
OptionalHeader.BaseOfData=0x2000
OptionalHeader.ImageBase=0x400000

### SECTION INFORMATION ###
1번째 section: CODE
PointerToRawData: 0x600
SizeOfRawData: 0x200
VirtualAddress: 0x1000
VirtualSize: 0x1000

2번째 section: DATA
PointerToRawData: 0x800
SizeOfRawData: 0x200
VirtualAddress: 0x2000
VirtualSize: 0x1000

3번째 section: .idata
PointerToRawData: 0xa00
SizeOfRawData: 0x200
VirtualAddress: 0x3000
VirtualSize: 0x1000

4번째 section: .reloc
PointerToRawData: 0xc00
SizeOfRawData: 0x200
VirtualAddress: 0x4000
VirtualSize: 0x1000

5번째 section: .rsrc
PointerToRawData: 0xe00
SizeOfRawData: 0x400
VirtualAddress: 0x5000
VirtualSize: 0x1000

### IAT ###
IAT가 저장된 섹션: .idata
RVA to RAW: 0x3000->0xa00
ImportDescriptor[0].Name=KERNEL32.dll
- function name (RVA=0x307c), GetDriveTypeA
- function name (RVA=0x308c), ExitProcess
ImportDescriptor[1].Name=USER32.dll
- function name (RVA=0x309a), MessageBoxA

C:\Users\백다현\source\repos\PEParser\Debug\PEParser.exe(프로세스 17912)이 (가) 0 코드(0x0)와 함께 종료되었습니다.
이 창을 닫으려면 아무 키나 누르세요...

```

## 2. 소스코드 #2 ( **PEParser\_2.cpp** )

```

#include <Windows.h>
#include <stdio.h>

int main(void)
{
    wchar_t path_pefile[] = L"C:\\Users\\백다현\\Desktop\\abex_crackme1.exe

```

```

\\abex_crackme1.exe";

HANDLE hFile = NULL, hFileMap = NULL;
LPBYTE lpFileBase = NULL;
DWORD dwSize = 0;

PIMAGE_DOS_HEADER pDosHeader = NULL;
PIMAGE_NT_HEADERS pNtHeader = NULL;

hFile = CreateFileW(path_pefile, GENERIC_READ, FILE_SHARE_READ, NU
LL, OPEN_EXISTING, FILE_ATTRIBUTE_NORMAL, NULL);
if (hFile == INVALID_HANDLE_VALUE)
{
    printf("CreateFileW() failed. Error code=%lu\n", GetLastError());
    return GetLastError();
}
dwSize = GetFileSize(hFile, 0);
printf("File size=%lu bytes\n\n", dwSize);

hFileMap = CreateFileMapping(hFile, NULL, PAGE_READONLY, 0, 0, NUL
L);
// lpFileBase 포인터는 OS에 의해 메모리에 로드된 PE 파일의 가장 첫 바이트를
가리킴.
//lpFileBase가 NULL이라면 MapViewOfFile() 함수가 실패했다는 의미이다. 여
기서는 예외 처리를 생략한다.
lpFileBase = (LPBYTE)MapViewOfFile(hFileMap, FILE_MAP_READ, 0, 0, d
wSize);

printf("File signature=%c%c\n", lpFileBase[0], lpFileBase[1]);

pDosHeader = (PIMAGE_DOS_HEADER)lpFileBase;
printf("Offset to the NT header=%#x\n\n", pDosHeader->e_lfanew);

pNtHeader = (PIMAGE_NT_HEADERS)(lpFileBase + pDosHeader->e_lfan
ew);
printf("OptionalHeader.BaseOfCode=%#x\n", pNtHeader->OptionalHead
er.BaseOfCode);

```

```

    printf("OptionalHeader.SizeOfCode= %#x\n", pNtHeader->OptionalHeader.SizeOfCode);
    printf("OptionalHeader.AddressOfEntryPoint= %#x\n", pNtHeader->OptionalHeader.AddressOfEntryPoint);
    printf("OptionalHeader.BaseOfData= %#x\n", pNtHeader->OptionalHeader.BaseOfData);
    printf("OptionalHeader.ImageBase= %#x\n\n", pNtHeader->OptionalHeader.ImageBase);

    printf("### SECTION INFORMATION ###\n");
    /*TODO: 여기서부터 코딩 시작*/
    // -----
    // [1] 섹션 테이블 출력
    // -----
    PIMAGE_FILE_HEADER pFileHeader = &pNtHeader->FileHeader;
    WORD numberOfSections = pFileHeader->NumberOfSections;

    // IMAGE_FIRST_SECTION(pNtHeader) 대신 수동 계산
    PIMAGE_SECTION_HEADER pSection = (PIMAGE_SECTION_HEADER)(
        (LPBYTE)&pNtHeader->OptionalHeader + pNtHeader->FileHeader.SizeOfOptionalHeader
    );

    for (WORD i = 0; i < numberOfSections; ++i) {
        char name[9] = { 0 };
        memcpy(name, pSection[i].Name, 8);

        printf("%d번째 section: %s\n", i + 1, name);
        printf("PointerToRawData: %#x\n", pSection[i].PointerToRawData);
        printf("SizeOfRawData: %#x\n", pSection[i].SizeOfRawData);
        printf("VirtualAddress: %#x\n", pSection[i].VirtualAddress);
        printf("VirtualSize: %#x\n\n", pSection[i].Misc.VirtualSize);
    }

    // -----
    // [2] IAT(Import Directory) 파싱
    // -----
    printf("### IAT ###\n");

```

```

IMAGE_DATA_DIRECTORY importDir =
    pNtHeader→OptionalHeader.DataDirectory[IMAGE_DIRECTORY_ENTRY_IMPORT]; // index 1

if (importDir.VirtualAddress == 0 || importDir.Size == 0) {
    printf("No Import Directory found.\n");
}
else {
    // (a) IAT가 저장된 섹션 찾기 + RVA→RAW 계산(인라인)
    int importRaw = -1;
    char containingSection[9] = "Unknown";
    for (WORD i = 0; i < numberOfSections; ++i) {
        DWORD va = pSection[i].VirtualAddress;
        DWORD vsize = pSection[i].Misc.VirtualSize ? pSection[i].Misc.VirtualSize : pSection[i].SizeOfRawData;
        if (importDir.VirtualAddress >= va && importDir.VirtualAddress < va + vsize) {
            importRaw = (int)(pSection[i].PointerToRawData + (importDir.VirtualAddress - va));
            memset(containingSection, 0, sizeof(containingSection));
            memcpy(containingSection, pSection[i].Name, 8);
            break;
        }
    }
    // 헤더 내부면 RAW == RVA
    if (importRaw < 0 && importDir.VirtualAddress < pNtHeader→OptionalHeader.SizeOfHeaders) {
        importRaw = (int)importDir.VirtualAddress;
        strcpy_s(containingSection, sizeof(containingSection), "Headers");
    }

    printf("IAT가 저장된 섹션: %s\n", containingSection);
    if (importRaw >= 0)
        printf("RVA to RAW: %#x→0x%x\n", importDir.VirtualAddress, importRaw);
    else
        printf("RVA to RAW 변환 실패 (IAT 위치 계산 불가)\n");
}

```

```

    if (importRaw >= 0 && (DWORD)importRaw < dwSize) {
        PIMAGE_IMPORT_DESCRIPTOR pDesc = (PIMAGE_IMPORT_DESCRIPTOR)(lpFileBase + importRaw);

        // (b) ImportDescriptor 배열 순회 (모두 0이면 종료)
        for (int idx = 0; ; ++idx, ++pDesc) {
            SIZE_T curOff = (SIZE_T)((LPBYTE)pDesc - lpFileBase);
            if (curOff + sizeof(IMAGE_IMPORT_DESCRIPTOR) > dwSize) break;

            if (pDesc->OriginalFirstThunk == 0 &&
                pDesc->FirstThunk == 0 &&
                pDesc->Name == 0 &&
                pDesc->TimeStamp == 0 &&
                pDesc->ForwarderChain == 0) {
                break;
            }

            // DLL 이름: Name(RVA) → RAW
            int nameRaw = -1;
            if (pDesc->Name < pNtHeader->OptionalHeader.SizeOfHeaders) {
                nameRaw = (int)pDesc->Name;
            }
            else {
                for (WORD i = 0; i < numberOfSections; ++i) {
                    DWORD va = pSection[i].VirtualAddress;
                    DWORD vsize = pSection[i].Misc.VirtualSize ? pSection[i].Misc.VirtualSize : pSection[i].SizeOfRawData;
                    if (pDesc->Name >= va && pDesc->Name < va + vsize) {
                        nameRaw = (int)(pSection[i].PointerToRawData + (pDesc->Name - va));
                        break;
                    }
                }
            }

            char dllName[512] = { 0 };

```

```

        if (nameRaw >= 0 && (DWORD)nameRaw < dwSize)
            strncpy_s(dllName, sizeof(dllName), (const char*)(IpFileBase +
nameRaw), _TRUNCATE);
        else
            strcpy_s(dllName, sizeof(dllName), "<invalid>");
        printf("ImportDescriptor[%d].Name=%s\n", idx, dllName);

        // (c) Thunk 테이블: OriginalFirstThunk 우선, 없으면 FirstThunk
        DWORD thunkRVA = pDesc->OriginalFirstThunk ? pDesc->OriginalFirstThunk : pDesc->FirstThunk;

        // thunkRVA → RAW
        int thunkRaw = -1;
        if (thunkRVA < pNtHeader->OptionalHeader.SizeOfHeaders) {
            thunkRaw = (int)thunkRVA;
        }
        else {
            for (WORD i = 0; i < numberOfSections; ++i) {
                DWORD va = pSection[i].VirtualAddress;
                DWORD vsize = pSection[i].Misc.VirtualSize ? pSection[i].Misc.VirtualSize : pSection[i].SizeOfRawData;
                if (thunkRVA >= va && thunkRVA < va + vsize) {
                    thunkRaw = (int)(pSection[i].PointerToRawData + (thunkRVA - va));
                    break;
                }
            }
        }

        if (thunkRaw < 0 || (DWORD)thunkRaw >= dwSize) {
            printf(" Thunk RVA → RAW 변환 실패\n");
            continue;
        }

        // (d) 개별 thunk 순회 (32-bit: 4바이트, 0이면 종료)
        SIZE_T off = (SIZE_T)thunkRaw;
        while (off + 4 <= dwSize) {
            DWORD entry = 0;

```

```

memcpy(&entry, lpFileBase + off, 4);
if (entry == 0) break;

if (entry & 0x80000000u) {
    WORD ord = (WORD)(entry & 0xFFFF);
    printf("- ordinal: %u\n", ord);
}
else {
    // entry는 IMAGE_IMPORT_BY_NAME 구조체의 RVA
    int ibnRaw = -1;
    if (entry < pNtHeader->OptionalHeader.SizeOfHeaders) {
        ibnRaw = (int)entry;
    }
    else {
        for (WORD i = 0; i < numberOfSections; ++i) {
            DWORD va = pSection[i].VirtualAddress;
            DWORD vsize = pSection[i].Misc.VirtualSize ? pSection
[i].Misc.VirtualSize : pSection[i].SizeOfRawData;
            if (entry >= va && entry < va + vsize) {
                ibnRaw = (int)(pSection[i].PointerToRawData + (entry
- va));
                break;
            }
        }
    }

    if (ibnRaw >= 0 && (DWORD)(ibnRaw + 2) < dwSize) {
        // [2바이트 hint] 다음이 함수명(ASCII, null-terminated)
        const char* fname = (const char*)(lpFileBase + ibnRaw +
2);

        printf("- function name (RVA=%#x), %s\n", entry, fname);
    }
    else {
        printf("- function name (RVA=%#x), <cannot read>\n", ent
ry);
    }
}
off += 4;

```

```

    }
    }
}

/*TODO: 여기까지 코딩*/

/*Windows로부터 할당받은 리소스를 역순으로 반환*/
UnmapViewOfFile(lpFileBase);
CloseHandle(hFileMap);
CloseHandle(hFile);
/*main() 함수가 끝까지 실행되었음을 알리기 위해 0을 반환*/
return 0;
}

```

```
Microsoft Visual Studio 디버그 x + v
File size=8192 bytes
File signature=MZ
Offset to the NT header=0x100
OptionalHeader.BaseOfCode=0x1000
OptionalHeader.SizeOfCode=0x200
OptionalHeader.AddressOfEntryPoint=0x1000
OptionalHeader.BaseOfData=0x2000
OptionalHeader.ImageBase=0x400000
### SECTION INFORMATION ###
1번째 section: CODE
PointerToRawData: 0x600
SizeOfRawData: 0x200
VirtualAddress: 0x1000
VirtualSize: 0x1000
2번째 section: DATA
PointerToRawData: 0x800
SizeOfRawData: 0x200
VirtualAddress: 0x2000
VirtualSize: 0x1000
3번째 section: .idata
PointerToRawData: 0xa00
SizeOfRawData: 0x200
VirtualAddress: 0x3000
VirtualSize: 0x1000
4번째 section: .reloc
PointerToRawData: 0xc00
SizeOfRawData: 0x200
VirtualAddress: 0x4000
VirtualSize: 0x1000
5번째 section: .rsrc
PointerToRawData: 0xe00
SizeOfRawData: 0x400
VirtualAddress: 0x5000
VirtualSize: 0x1000
### IAT ###
IAT가 저장된 섹션: .idata
RVA to RAW: 0x3000->0xa00
ImportDescriptor[0].Name=KERNEL32.dll
- function name (RVA=0x307c), GetDriveTypeA
- function name (RVA=0x308c), ExitProcess
ImportDescriptor[1].Name=USER32.dll
- function name (RVA=0x309a), MessageBoxA
C:\Users\백다현\source\repos\PEParser_2\Debug\PEParser_2.exe(프로세스 5760)이(가) 0 코드(0x0)와 함께 종료되었습니다.
이 창을 닫으려면 아무 키나 누르세요...
```